CITS4402 Computer Vision PROJECT

Title: Face detection and matching
Total marks: 100
Project weight in the final unit marks: 25%

Grouping and Submission

- Form groups of **two** to **three** members.
- Each group must submit **one** project report along with code via LMS by the deadline.
- All group members will receive the **same** mark, assuming equal contribution.

Project Overview and marks

Face detection and facial analysis are fundamental tasks in computer vision with applications in access control (e.g. phone passcode), surveillance, fatigue detection, customer service and medical diagnosis. In this project, you will write a program that performs the following tasks:

1. **Graphical User Interface along with project report [10 marks]:** Make a GUI that should display the input image on the left and the output image on the right. It should also display messages for processing time and the results. The GUI must have two buttons
    a. Button A (Single Image): Press to select and load an image. It should also display the image in GUI and then automatically process it and display the result (step 4) in the GUI too.
    b. Button B (Bulk Processing): Press to select a folder that contains input images for bulk processing. It should load and process the images one by one and save the results in a sub-folder and display the identity clustering results on the GUI (step 5).

2. **Face Detection [10 + 5 marks]:** Detect faces in an input image and show a box around each.
    a. Input will be coloured images of size 640 x 480
    b. Assume 1 to 4 faces in an image

3. **Facial Landmark Detection [15 + 5 marks]:** Detect three facial landmarks (right eye-centre, left eye-centre, nose tip) for each detected face.

4. **Facial Alignment and display [20 + 5 marks]:** Align, crop and resize each detected face to 125 x 125 size and display it on top of the original image at one of the corners. You need to figure out the order in which the three operations (align, crop, resize) should be performed to get the desired output.
    a. Align/resize each face so that the eyes are at location (40,40) and (85,40) and the nose tip is at location (63,70).
    b. Each aligned-resized 125 x 125 image should be displayed on the output image so that they occupy the corners (top-left, top-right, bottom-left, bottom-right). Any order is acceptable.
    c. Highlight landmarks on each face using red, green blue circles (right-eye, left-eye, nose-tip). Highlight them on the original face as well as those displayed on the image corner.
    d. Refer to the two sample figures at the end to see how the output image should look like. Display similar messages on the screen when a single image is processed.

5. **Identity clustering [20 + 10 marks]:** Bulk process all images in the specified folder (Button B), extract the detected faces from each image and cluster them to find the number of unique identities. For

example, your code processed 10 images and there were a total 24 faces detected but these 24 faces belong to only 6 distinct people. Your code must:

a. Read images from one folder
b. Save the bulk processed images (item c below) to a new folder called "Processed_Images" at the same location as the original folder containing the face images.
c. Bulk processed output images should be the cropped aligned images of size 125 x 125 (without landmarks). These images are to be saved with the filename format "Identity_N_face_M.jpg" (Or any image format such as .png)
d. Display the clustering results as text on GUI e.g. "Total X images processed in S seconds. Y faces detected corresponding to Z unique identities."
e. If "Processed_Images" folder does not exist, your code should make a folder by this name. If it exists, your code should delete all files inside that folder before saving the new results to the folder so that results from two different runs of bulk processing do not mix.

NOTE: You are given 10 sample images to try your code on. The GUI and performance on these 10 images carry 75 marks. Your code will also be tested on 10 hidden images of similar complexity, for which the remaining 25 marks are reserved (+5 marks in item 2 to 4 and +10 marks in item 5). Full +5/+10 marks (+5 in items 2 to 4 and +10 in item 5 = 25 marks) for over 90% accuracy on the hidden images. +4/+8 marks for 80-90% accuracy, +3/+6 marks for 70-80% accuracy, +2/+4 marks for 60-70% accuracy and +1/+2 marks for 50-60% accuracy. If your code has less than 50% accuracy on the hidden images, you get 0/25 marks.

Project Resources and Tips

1. GUI: Use tkinter and the lab starter code to make the GUI. Label the buttons, image displays (e.g. "Input Image", "Processed Image") and output texts appropriately. The GUI marks are for functionality and not for how beautiful it looks. The names of the buttons and the wording of the display messages do not need to match exactly as specified above but should contain the required information.

2. You can use any library for face detection e.g. a pretrained deep learning-based face detector for colour images. However, you must also use skin colour segmentation to assist the face detector in some way e.g. to remove false positives or to locate possible regions of faces. This part may not improve the accuracy of face detection but is intended to test your understanding of colour features. It must be implemented separately before or after the face detector.

3. You can use a pretrained (library function) for facial landmark detection too. You must ensure that the landmarks are detected inside the face windows (from step 2) and not on the complete image (as that would be redundant).

4. You must write code for this part. You will need to apply similarity transformation to align face landmarks to the 3 specified target landmarks. You must choose the right sequence of sub-steps so that the aligned (rotated) image does not end up having blank regions near the borders nor is the face region cut out.

5. This step simply repeats step 2 to 4 for many images and then applies clustering on the detected faces. For clustering you first need to compute features for each face. You are allowed to use a pre-trained model for this. Clustering must find the number of optimal clusters automatically which will

be the number of unique identities. Hence k-means type clustering method where you need to specify the number of clusters cannot be directly applied here. You can use DBSCAN or any other clustering method that does not require you to specify the cluster numbers (because that is what you want to automatically find).

6. Data: You are free to find any dataset to test your code on or train/fine-tune your models. The 10 sample images are provided as a guide. Your submitted code will be tested on these 10 images and another 10 hidden images of similar difficulty.

7. OPTIONAL part (No marks): Display the clustering results graphically. Project the face features to PCA space and display the top two PCs on a 2-D plot. Give different colour to each cluster and display the best example face next to its corresponding cluster.

Project Demonstration: One member from each team is required to demonstrate the project to a lab facilitator in Week 12. At a minimum, the first four functionalities should be fully operational, even if the fifth is still under development

---

Submission Requirements

- Ensure your code is **well-documented** with clear comments.

- Use **meaningful variable names** and follow **best coding practices**.

- At the very beginning of your report and script file, note down the **full name** and **student number** of each group member.

- Submit a single zip file containing a directory with the following files:
    - **LastNameSt-1_LastNameSt-2_LastNameSt-3_Report.pdf.** A three page project report (minimum font size 11) where you explain your reasons for choosing a specific method, library, and model, how you optimized any hyperparameters and what other alternatives are available.

    - **LastNameSt-1_LastNameSt-2_LastNameSt-3_Script.py**. Your script file that contains all your code for full functionality including launching the GUI. Your script file must be fully contained.

    - **Any trained deep learning models used by your script**

    - Any other file required by your script

- If you have questions, please come to the lab and discuss it with the lab facilitators.

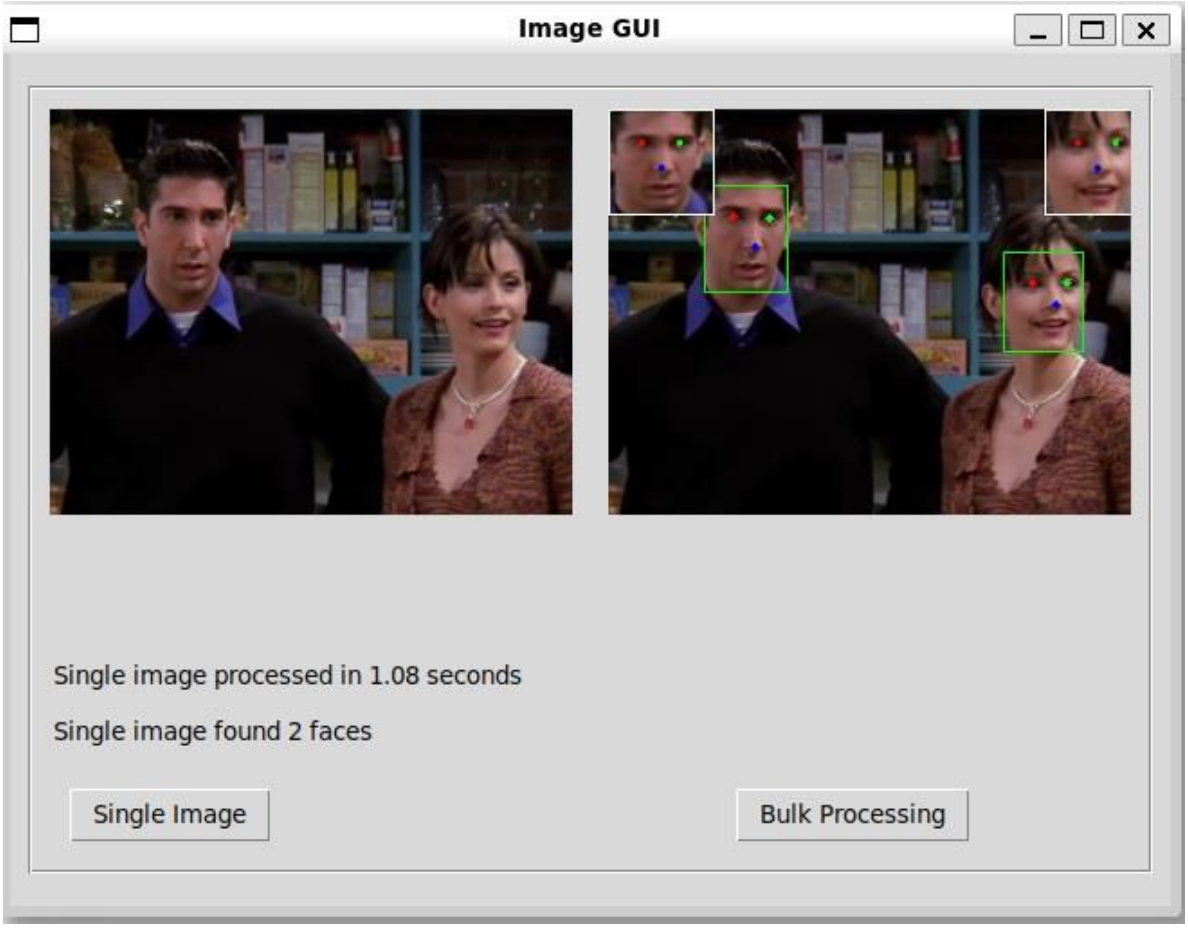
Image GUI
Single image processed in 1.08 seconds
Single image found 2 faces
Single Image
Bulk Processing

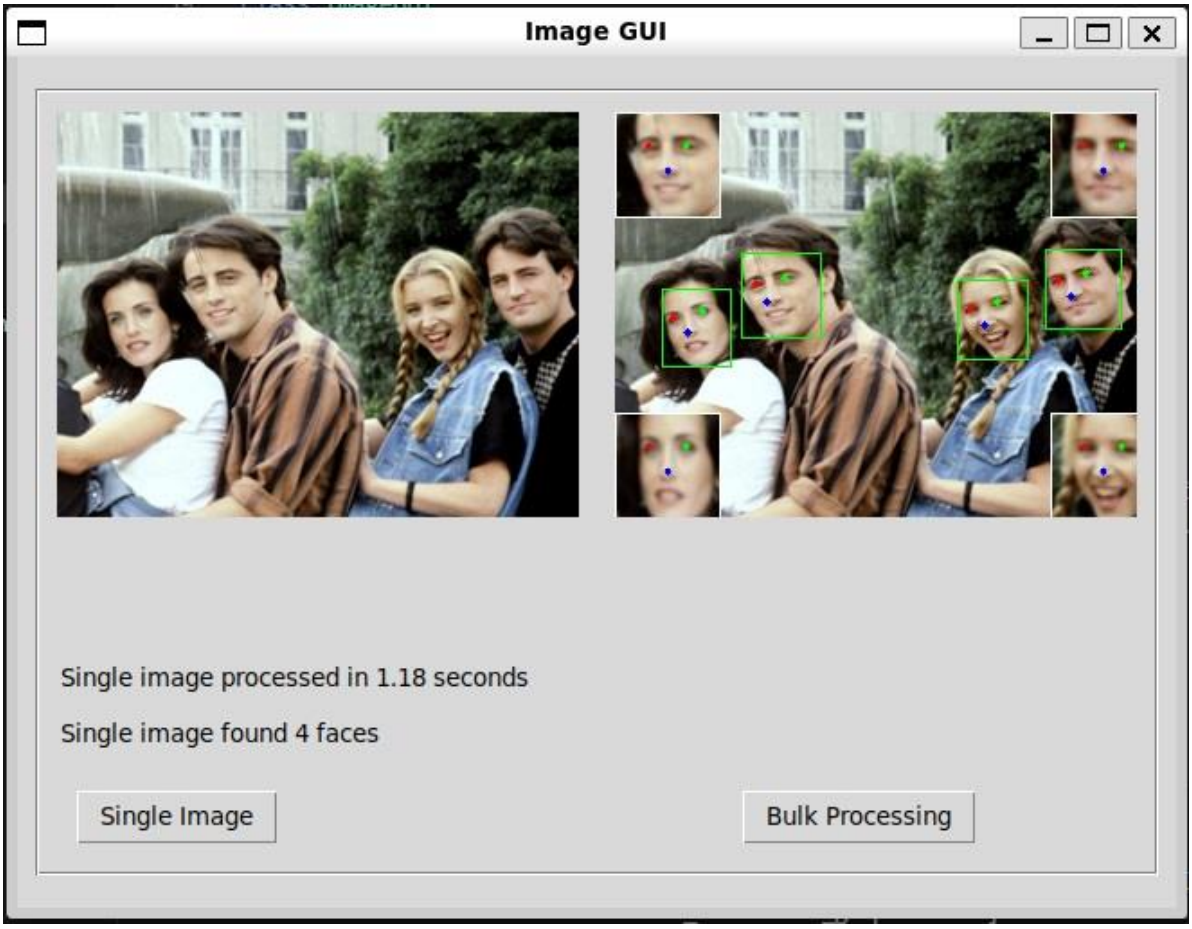
Image GUI
Single image processed in 1.18 seconds
Single image found 4 faces
Single Image
Bulk Processing